

Manipulando texto com Python

Trabalhando com Texto em Python I · Unidade 1

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Computadores e texto	1
2. Codificação de texto (<i>text encoding</i>)	2
3. Carregando arquivos de texto simples no Python	3
4. Manipulando texto	5
5. Manipulando texto em escala	8
Quiz	12
Resumo da unidade	13

Objetivos de aprendizagem

Ao final desta unidade, você deverá ser capaz de:

- **compreender a diferença** entre texto formatado (*rich text*), texto estruturado (*structured text*) e texto simples (*plain text*);
- **entender o conceito** de codificação de caracteres (*text encoding*);
- **carregar arquivos** de texto simples no Python e **manipular** o seu conteúdo;
- **definir padrões flexíveis** de busca com expressões regulares;
- **processar vários arquivos** de uma só vez e salvar os resultados.

1. Computadores e texto

Os computadores podem armazenar e representar texto em formatos diferentes. Conhecer a distinção entre esses tipos é **fundamental** para processá-los programaticamente, porque cada formato traz consigo informações (ou ruídos) distintos.

1.1 O que é texto formatado (*rich text*)?

Processadores de texto, como o Microsoft Word, produzem **texto formatado** (*rich text*): texto cuja aparência foi estilizada de alguma maneira específica.

O texto formatado permite definir estilos visuais para os elementos do documento. Títulos, por exemplo, podem usar uma fonte diferente da do corpo do texto, que por sua vez pode empregar itálico ou negrito para dar ênfase. O texto formatado também pode incluir imagens, tabelas e outros elementos.

Esse é o formato padrão dos processadores de texto modernos do tipo *WYSIWYG* (*what you see is what you get* — “o que você vê é o que você obtém”).

1.2 O que é texto simples (*plain text*)?

Diferentemente do texto formatado, o **texto simples** (*plain text*) não contém nenhuma informação sobre a aparência visual: ele é feito **apenas de caracteres**.

Neste contexto, “caracteres” abrange letras, números, sinais de pontuação, espaços e quebras de linha. A definição de texto simples é um tanto flexível, mas, em geral, refere-se a texto sem qualquer informação de formatação ou estilo.

1.3 O que é texto estruturado (*structured text*)?

O **texto estruturado** (*structured text*) pode ser entendido como um caso especial de texto simples que inclui sequências de caracteres usadas para **formatar o texto para exibição**.

São exemplos de texto estruturado os textos descritos por linguagens de marcação como XML, Markdown ou HTML.

O exemplo abaixo mostra uma frase em texto simples envolvida por marcadores (*tags*) HTML de parágrafo `<p>`. A marca de abertura `<p>` e a de fechamento `</p>` informam ao computador que todo o conteúdo colocado entre elas forma um parágrafo:

```
<p>This is an example sentence.</p>
```

Essa informação é usada para estruturar o texto simples no momento de renderizá-lo para exibição, normalmente aplicando estilo à sua aparência.

1.4 Por que isso importa?

Ao reunir um conjunto de textos para formar um **corpus**, é bem provável que parte deles tenha origem em formato formatado ou estruturado, dependendo do meio de onde vieram:

- Se você coleta documentos impressos que foram digitalizados por **reconhecimento óptico de caracteres** (*optical character recognition*, OCR) e depois convertidos de texto formatado para texto simples, a remoção das informações de formatação tende a **introduzir erros** no texto resultante. Trabalhar com esse tipo de OCR “sujo” pode afetar os resultados da análise textual (Hill & Hengchen, 2019).
- Se você coleta documentos digitais raspando fóruns de discussão ou sites, é provável que encontre vestígios de texto estruturado na forma de marcadores de marcação, que podem ser arrastados para o texto simples durante a conversão.

O texto simples é, de longe, o formato **mais interoperável** para texto, por ser fácil de ler para os computadores. É por isso que as linguagens de programação trabalham com texto simples — e, se você pretende usar programação para manipular texto, precisa saber o que é texto simples.

Em resumo: ao trabalhar com texto simples, muitas vezes será preciso lidar com os vestígios deixados pela conversão a partir de texto formatado ou estruturado.

2. Codificação de texto (*text encoding*)

Para serem lidos pelos computadores, os textos simples precisam ser **codificados**. Isso é feito por meio da **codificação de caracteres** (*character encoding*), que mapeia cada caractere (letras, números, pontuação, espaços. . .) para uma representação numérica compreendida pela máquina.

O ideal seria não termos de lidar com operações de baixo nível como a codificação de caracteres — mas, na prática, precisamos, porque existem **vários sistemas** de codificação e eles **não são compatíveis entre si**. Essa é a origem de boa parte das dores de cabeça de quem trabalha com texto simples.

Há dois sistemas de codificação com os quais você provavelmente vai se deparar: **ASCII** e **Unicode**.

2.1 ASCII

O **ASCII** (*American Standard Code for Information Interchange*) é um sistema pioneiro de codificação de caracteres que serviu de base para muitos sistemas modernos.

O ASCII ainda é bastante usado, mas é **muito limitado** quanto à variedade de caracteres. Se o seu idioma inclui caracteres como *ä, ö* — ou os nossos *á, ç, ã* —, o ASCII não dá conta.

2.2 Unicode

O **Unicode** é um padrão para codificar texto na maioria dos sistemas de escrita usados no mundo, cobrindo cerca de **140 mil caracteres** entre escritas modernas e históricas, símbolos e emojis.

Por exemplo, o emoji de fatia de pizza tem o “código” Unicode **U+1F355**, enquanto o código correspondente a um espaço em branco é **U+0020**.

O Unicode pode ser implementado por diferentes codificações, como o **UTF-8**, definido pelo próprio padrão Unicode.

O UTF-8 é **retrocompatível com o ASCII**. Em outras palavras, as codificações do ASCII formam um subconjunto do UTF-8 — o que facilita bastante a nossa vida. Assim, mesmo que um arquivo de texto simples tenha sido codificado em ASCII, podemos decodificá-lo como UTF-8; **o contrário, porém, não vale**.

3. Carregando arquivos de texto simples no Python

Arquivos de texto simples podem ser carregados no Python com a função `open()`.

O primeiro argumento de `open()` deve ser uma *string* contendo o **caminho** para o arquivo que será aberto. Aqui, temos um caminho que aponta para um arquivo chamado `NYT_1991-01-16-A15.txt`, localizado em um diretório chamado `data`. Na definição do caminho, o diretório e o nome do arquivo são separados por uma barra `/`.

Para acessar o arquivo apontado por esse caminho, passamos o caminho como *string* ao argumento `file` da função `open()`:

```
open(file='data/NYT_1991-01-16-A15.txt', mode='r', encoding='utf-8')
```

Vale entender os demais argumentos:

- Por padrão, o Python 3 assume que o texto está codificado em **UTF-8**, mas podemos deixar isso explícito com o argumento `encoding`, passando a *string* `utf-8`.

- O argumento `mode` define o que queremos fazer com o arquivo. A *string* `r` (de *read*) indica que queremos **apenas ler** o arquivo.
- Usamos `open()` junto da instrução `with`, que garante que o arquivo será **fechado** após executarmos o que precisamos dentro do bloco de código indentado que segue o `with`. Isso evita que o arquivo continue consumindo memória e recursos depois que não precisamos mais dele.

```
# Abre um arquivo e o atribui à variável 'file'
with open(file='data/NYT_1991-01-16-A15.txt', mode='r', encoding='utf-8') as file:

    # A instrução 'with' deve ser seguida por um bloco de código indentado.
    # Aqui chamamos o método read() para ler o conteúdo do arquivo e
    # atribuímos o resultado à variável 'text'.
    text = file.read()
```

Repare que `with` e `open()` são seguidos da palavra `as` e de uma variável chamada `file`. Isso instrui o Python a atribuir à variável `file` aquilo que `open()` retornar.

Se agora chamarmos a variável `file`, obtemos um objeto Python do tipo `TextIOWrapper`, que carrega três informações: o caminho do arquivo (no argumento `name`) e os argumentos `mode` e `encoding` que definimos acima:

```
# Chama a variável para examinar o objeto
file
```

```
<_io.TextIOWrapper name='data/NYT_1991-01-16-A15.txt' mode='r' encoding='utf-8'>
```

Tenha em mente que, no bloco indentado após o `with`, chamamos o método `read()` do objeto `TextIOWrapper`. Esse método leu o conteúdo do arquivo, que atribuímos à variável `text`.

No entanto, se tentarmos chamar `read()` para a variável `file` **fora** do bloco `with`, o Python levanta um erro, porque o arquivo já foi fechado:

```
# Tenta usar o método read() para ler o conteúdo do arquivo
file.read()
```

```
ValueError: I/O operation on closed file.
```

Esse comportamento é o esperado: queremos que o arquivo seja fechado, para que não consuma memória nem recursos quando não é mais necessário. Isso é especialmente importante ao trabalhar com **milhares de arquivos**, pois cada arquivo aberto ocupa memória.

Vejamos agora o resultado de `read()`, guardado em `text`. O texto é bem longo, então vamos pegar apenas uma **fatia** com os primeiros 500 caracteres, usando colchetes `[ : 500 ]`. Adicionar colchetes logo após o nome de uma variável permite acessar partes do objeto, quando o objeto permite isso. Por exemplo, `text[1]` recuperaria o caractere na posição 1; o `:` antes do número instrui o Python a recuperar **todos** os caracteres até o de posição 500.

```
# Recupera os primeiros 500 caracteres da variável 'text'
text[:500]
```

```
'U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. Is Ordered to Find Iraqis Whose Visas Have
↳ Expired\nBy JAMES BARRON\nNew York Times (1923-Current file); Jan 16, 1991; ... the
↳ Justice Department'
```

A maior parte do texto está legível, mas há sequências estranhas, como `bem` no início e vários `\n` ao longo do texto:

- A sequência (código `U+FEFF`) é o **BOM** (*byte order mark*, “marca de ordem de bytes”). Ele foi criado para indicar a ordem dos bytes em codificações como UTF-16; **em UTF-8 não há ordem de bytes**, então o BOM é **desnecessário** e seu uso é **desaconselhado** pelo próprio padrão Unicode. Ainda assim, alguns programas e editores (especialmente no Windows) o adicionam no início do arquivo como uma espécie de “assinatura” de UTF-8. É por isso que ele aparece aqui como um caractere fantasma no começo do texto — e, se não for removido, pode causar bugs (por exemplo, a primeira palavra deixa de casar em comparações). Nem todo arquivo UTF-8 contém essa marca.
- As sequências `\n` indicam **quebra de linha**.

Isso fica evidente se usarmos a função `print()` para imprimir os primeiros 1000 caracteres:

```
# Imprime os primeiros 1000 caracteres da variável 'text'
print(text[:1000])
```

```
U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. Is Ordered to Find Iraqis Whose Visas Have
→ Expired
By JAMES BARRON
New York Times (1923-Current file); Jan 16, 1991;
ProQuest Historical Newspapers: The New York Times with Index pg. A15
...
    The Federal Bureau of Investigation has been ordered to track down as many as 3,000
→ Iraqis in this country whose visas have expired, the Justice Department said
→ yesterday.
```

Como se vê, o Python sabe interpretar as sequências `\n` e insere uma quebra de linha ao encontrá-las durante a impressão. Note também que as primeiras linhas do arquivo contêm **metadados** do artigo (nome, autor e fonte), que antecedem o corpo do texto.

4. Manipulando texto

Como todo o conteúdo guardado em `text` é um objeto *string*, podemos usar todos os métodos de manipulação de *strings* do Python.

4.1 Substituindo com `replace()`

Vamos usar o método `replace()` para trocar todas as quebras de linha `"\n"` por *strings* vazias `""` e guardar o resultado em `processed_text`:

```
# Substitui as quebras de linha \n por strings vazias e atribui o resultado
# à variável 'processed_text'
processed_text = text.replace('\n', '')

# Imprime os primeiros 1000 caracteres da variável 'processed_text'
print(processed_text[:1000])
```

Agora todo o texto ficou “grudado”. Ainda assim, conseguimos identificar o início de cada parágrafo, marcado por **três espaços em branco**.

Atenção: remover as quebras de linha também fez os metadados do artigo se fundirem em

um único parágrafo. Sempre fique atento aos **efeitos indesejados** de substituições e outras transformações!

4.2 Dividindo com `split()`

Se nos interessa apenas o corpo do artigo, podemos remover os metadados com facilidade, pois sabemos que eles estão separados do corpo por três espaços. A maneira mais simples é usar `split()` para **dividir a string em uma lista**, usando os três espaços como separador:

```
# Usa o método split() com três espaços como separador. Atribui o
# resultado à variável 'processed_text'.
processed_text = processed_text.split(sep='  ')

# Imprime o resultado da variável 'processed_text'
print(processed_text)
```

O método `split()` retorna uma **lista** de objetos *string*. Podemos confirmar isso verificando o tipo do objeto:

```
# Verifica o tipo do objeto na variável 'processed_text'
type(processed_text)
```

```
list
```

Os metadados ficaram no **primeiro item** da lista, pois a primeira sequência de três espaços aparece justamente onde os metadados terminam. Vamos buscar esse primeiro item — lembre que o Python começa a contar do zero, então ele está no índice **0**:

```
# Recupera o objeto string no índice 0 da lista 'processed_text'
processed_text[0]
```

4.3 Removendo itens com `pop()`

Para remover os metadados e manter apenas o corpo do texto, usamos o método `pop()` da lista. Ele espera um número inteiro: o índice do item a ser removido.

```
# Chama o método pop() da lista 'processed_text' com o
# índice do item a ser removido.
processed_text.pop(0)
```

Você pode se perguntar por que **não** atribuímos o resultado a uma variável. A resposta é que as listas do Python são **mutáveis**, ou seja, podem ser alteradas “no lugar” (*in place*). O método `pop()` modifica a lista sem precisar reatribuir o valor à variável. Podemos conferir recuperando os três primeiros itens:

```
# Recupera os três primeiros itens da lista 'processed_text'
processed_text[:3]
```

O primeiro item da lista **não corresponde mais** aos metadados.

4.4 Reunindo com `join()`

Para converter a lista de volta em uma *string*, usamos o método `join()` de um objeto *string*. O `join()` espera um **iterável** como entrada (algo que possa ser percorrido, como uma lista ou um dicionário).

Aqui pode haver certa confusão: o `join()` deve ser chamado **sobre a *string* que servirá de “cola”** entre os itens do iterável. No nosso caso, queremos usar como cola a sequência original que separava os parágrafos — uma quebra de linha e três espaços (`'\n '`):

```
# Usa o método join() para unir os itens da lista 'processed_text'
# usando a string '\n   ' - uma quebra de linha e três espaços. Guarda o
# resultado na variável de mesmo nome.
processed_text = '\n   '.join(processed_text)

# Verifica o resultado imprimindo os primeiros 1000 caracteres da string
# resultante em 'processed_text'
print(processed_text[:1000])
```

Aplicar o `join()` devolve uma *string* com as quebras de parágrafo originais!

4.5 Um “pipeline” de substituições com `laço for`

Examinando o texto de perto, dá para ver resquícios da digitalização: o OCR resultou em uma **mistura de aspas** de tipos diferentes, como `"`, `'`, `”`, `‘` e `”`. Se quiséssemos extrair citações do corpo do texto, seria bom padronizar as aspas. Vamos escolher `"` (uma aspa dupla simples) como padrão.

Poderíamos aplicar `replace()` separadamente para cada tipo de aspa, mas isso seria tedioso. Para tornar o processo mais eficiente, combinamos duas estruturas de dados do Python: **listas** e **tuplas**.

Começamos definindo uma lista chamada `pipeline`. Criamos e preenchemos uma lista simplesmente colocando objetos entre colchetes `[]`, separados por vírgula. Como `replace()` recebe duas *strings*, combinamos cada par em uma **tupla** — estruturas finitas e ordenadas, marcadas por parênteses `()`. Em cada tupla, colocamos o caractere a ser substituído na primeira *string* e o substituto na segunda:

```
# Define uma lista com quatro tuplas, cada uma com duas strings: o caractere
# a ser substituído e o seu substituto.
pipeline = [('"', '"'), ('‘', '’'), ('”', '”'), ('’', '’')]
```

Isso ilustra como diferentes estruturas de dados costumam ser **aninhadas** no Python: a lista contém tuplas, e as tuplas contém *strings*.

Agora podemos percorrer cada item da lista com um `laço for`, que itera pelos itens na ordem em que aparecem. Cada item é uma tupla com duas *strings*. Para entrar no `laço`, o Python espera que a próxima linha esteja **indentada** (use a tecla `Tab` →):

```
# Percorre as tuplas da lista 'pipeline'. Cada tupla tem dois valores, que
# atribuímos as variáveis 'old' e 'new' automaticamente!
for old, new in pipeline:

    # Usa o método replace() para substituir a string da variável 'old'
    # pela string da variável 'new'
    processed_text = processed_text.replace(old, new)
```

O que acontece dentro do `laço` é exatamente o que fizemos antes com `replace()`, mas, em vez de definir manualmente as *strings*, usamos as *strings* contidas nas variáveis `old` e `new`! A cada volta, atualizamos automaticamente a *string* em `processed_text`.

```
# Imprime a string
print(processed_text)
```

Conseguimos realizar uma série de substituições percorrendo a lista de tuplas, que definia os padrões a substituir e seus substitutos.

Recapitulando a sintaxe do `for`: declaramos o início do laço com `for`, seguido de uma variável usada para se referir aos itens obtidos da lista. A lista percorrida vem precedida de `in` e do nome da variável atribuída à lista inteira.

Para entender melhor, vejamos o que acontece se definirmos **apenas uma** variável, `our_tuple`, para os itens obtidos:

```
# Percorre os itens da variável 'pipeline'
for our_tuple in pipeline:

    # Imprime o objeto retornado
    print(our_tuple)
```

```
('...', '...')
('...', '...')
('...', '...')
('...', '...')
```

Isso imprime as **tuplas** inteiras! O Python é esperto o bastante para entender que uma única variável se refere aos itens (as tuplas) da lista, ao passo que, com duas variáveis, ele avança até as *strings* contidas dentro de cada tupla. Ao escrever laços `for`, preste muita atenção aos itens contidos na lista!

5. Manipulando texto em escala

O ideal é que o Python permita manipular texto **em escala**, isto é, aplicar o mesmo procedimento a dez, cem ou mil arquivos com o mesmo esforço. Para isso, precisamos definir padrões **mais flexíveis** do que as *strings* fixas usadas com `replace()`, além de abrir e fechar arquivos automaticamente. Essas capacidades vêm dos módulos de **expressões regulares** e de **manipulação de arquivos**.

5.1 Expressões regulares (*regular expressions*)

As expressões regulares são uma “linguagem” que permite definir **padrões de busca**. Esses padrões podem ser usados para encontrar (ou encontrar e substituir) trechos em objetos *string*. Diferentemente das *strings* fixas, elas permitem definir **curingas** (caracteres que representam qualquer caractere), **quantificadores** (que casam sequências repetidas) e muito mais.

O Python oferece expressões regulares por meio do módulo `re`, que ativamos com o comando `import`:

```
import re
```

Vamos carregar um arquivo, ler seu conteúdo, atribuir os **últimos** 2000 caracteres à variável `extract` e imprimir o resultado:

```
# Define o caminho do arquivo e o abre para leitura (r) com codificação utf-8
with open(file='data/WP_1990-08-10-25A.txt', mode='r', encoding='utf-8') as file:

    # Lê o conteúdo do arquivo com o método .read()
    text = file.read()
```



```
# Pega os *últimos* 2000 caracteres - note o sinal de menos antes do número
extract = text[-2000:]

# Imprime o resultado
print(extract)
```

O texto tem muitos erros de OCR, principalmente sequências como `....` e `"""`.

Vamos **compilar** nossa primeira expressão regular, que busca sequências de dois ou mais pontos finais, com a função `compile()` do módulo `re`. Ela recebe uma *string* como entrada. Note o prefixo `r` antes da *string*: ele diz ao Python para guardar a *string* em formato “bruto” (*raw*), ou seja, exatamente como aparece.

```
# Compila uma expressão regular e a atribui a variável 'stops'
stops = re.compile(r'\.{2,}')
```

```
# Vamos verificar o tipo da expressão regular!
type(stops)
```

```
re.Pattern
```

Vamos destrinchar essa expressão:

- Ela é definida por uma *string* do Python (aspas simples `' '`).
- Precisamos de uma **barra invertida** `\` antes do ponto final `.`, porque, nas expressões regulares, o ponto final é um **curinga** que representa qualquer caractere. A barra diz ao Python que queremos o ponto final “de verdade”.
- As **chaves** `{ }` instruem a expressão a buscar ocorrências do item anterior (`\.`, nosso ponto de verdade) que aconteçam **duas ou mais vezes** (`2,`). Isso preserva os usos legítimos de um único ponto final.

Em português claro: buscamos ocorrências de **dois ou mais pontos finais**.

Para aplicar a expressão a algum texto, usamos o método `sub()` do objeto recém-criado `stops`. O `sub()` recebe dois argumentos:

- `repl`: a *string* usada para **substituir** as ocorrências encontradas;
- `string`: o objeto *string* onde procurar as ocorrências.

O método retorna a *string* modificada:

```
# Aplica a expressão regular ao texto em 'extract' e salva a saída
# na mesma variável, essencialmente sobrescrevendo o texto antigo.
extract = stops.sub(repl='', string=extract)
```

```
# Imprime o texto para examinar o resultado
print(extract)
```

As sequências de pontos finais desapareceram.

5.2 Alternativas na expressão regular

Podemos tornar a expressão mais poderosa acrescentando **alternativas**. Vamos compilar outra e guardá-la em `punct`:

```
# Compila uma expressão regular e a atribui a variável 'punct'
punct = re.compile(r'(\.|,){2,}')
```

A novidade são os **parênteses** () e a **barra vertical** | entre eles, que separa o ponto final \. da vírgula ,. Caracteres entre parênteses e separados por | marcam **alternativas**. Em português claro: buscamos ocorrências de **dois ou mais pontos finais ou vírgulas**.

Para garantir que o padrão funcione como esperado, recuperamos o texto original de `text` e o reatribuímos a `extract`, sobrescrevendo as edições anteriores:

```
# "Reinicia" a variável extract pegando os últimos 2000 caracteres da string original
extract = text[-2000:]

# Aplica a expressão regular
extract = punct.sub(repl='', string=extract)

# Imprime o resultado
print(extract)
```

Sucesso! As sequências de pontos finais e de vírgulas podem ser removidas com uma única expressão regular.

Sequências mais irregulares vindas de erros de OCR — como `'- '*`, `->."`, `/*-.` — são bem mais difíceis de capturar e exigiriam expressões mais complexas. Essa complexidade é justamente o que torna as expressões regulares tão poderosas, mas aprender a usá-las leva **tempo e paciência**.

Dica: use um serviço como regex101.com para aprender e testar expressões regulares interativamente.

Na prática, criar expressões que cubram o máximo de casos é particularmente difícil. Capturar a maioria dos erros — talvez distribuindo as transformações em uma **sequência de etapas** (um *pipeline*) — já ajuda muito a preparar o texto para análise. Lembre-se, porém: para identificar padrões de manipulação de forma confiável, você deve sempre observar **mais de um** texto do seu corpus.

5.3 Processando múltiplos arquivos

Muitos corpora contêm textos em vários arquivos. Para manipular grandes volumes de texto com eficiência, precisamos abrir os arquivos, ler seu conteúdo, executar as operações desejadas e fechá-los — tudo **programaticamente**.

Esse procedimento fica bem simples com a classe `Path` do módulo `pathlib` do Python. Usar `from ... import ...` permite importar **apenas parte** de um módulo — aqui, só a classe `Path`:

```
from pathlib import Path
```

A classe `Path` codifica informações sobre caminhos em uma estrutura de diretórios. O grande trunfo dela é **inferir automaticamente** o tipo de caminho usado pelo seu sistema operacional. Isso importa porque Windows, Linux e macOS usam caminhos de arquivo diferentes; a classe `Path` nos poupa de muita dor de cabeça, sobretudo se quisermos que o código rode em sistemas diferentes.

Nosso repositório contém um diretório chamado `data`, com os arquivos de texto que viemos usando. Vamos inicializar um objeto `Path` apontando para esse diretório e atribuí-lo à variável `corpus_dir`:

```
# Cria um objeto Path que aponta para o diretório 'data' e o atribui
# a variável 'corpus_dir'
corpus_dir = Path('data')
```

O objeto `Path` tem vários métodos e atributos úteis. Podemos, por exemplo, verificar se o caminho é válido com `exists()`, que retorna um valor **booleano** (`True` ou `False`):

```
# Usa o método exists() para verificar se o caminho é válido
corpus_dir.exists()
```

```
True
```

Também podemos checar se o caminho é um diretório com `is_dir()`, e garantir que ele **não** aponta para um arquivo com `is_file()`:

```
# Usa o método is_dir() para verificar se o caminho aponta para um diretório
corpus_dir.is_dir()
```

```
True
```

```
# Usa o método is_file() para verificar se o caminho aponta para um arquivo
corpus_dir.is_file()
```

```
False
```

Sabendo que o caminho aponta para um diretório, usamos o método `glob()` para coletar todos os arquivos de texto. O nome `glob` vem de *global* e foi implementado originalmente como um programa para casar nomes de arquivos e caminhos usando curingas. O `glob()` exige o argumento `pattern`, que recebe uma *string*: o `*` (asterisco) atua como **curinga**, representando qualquer sequência de caracteres antes de `.txt` (sufixo comum de arquivos de texto simples). Convertendo o resultado em lista com `list()`, podemos percorrê-lo facilmente:

```
# Coleta todos os arquivos com sufixo .txt no diretório 'corpus_dir' e converte o
# resultado em uma lista
files = list(corpus_dir.glob(pattern='*.txt'))

# Mostra o resultado
files
```

```
[PosixPath('data/WP_1990-08-10-25A.txt'),
 PosixPath('data/NYT_1991-01-16-A15.txt'),
 PosixPath('data/WP_1991-01-17-A1B.txt')]
```

Temos uma lista de três objetos `Path` apontando para três arquivos! Isso nos permite percorrê-los com um laço `for` e manipular o texto de cada um. No bloco abaixo, iteramos sobre cada arquivo, lemos e modificamos seu conteúdo e o gravamos em um novo arquivo:

```
# Percorre a lista de objetos Path em 'files'. Refere-se a cada arquivo
# pela variável 'file'.
for file in files:

    # Usa o método read_text() de um objeto Path para ler o conteúdo do arquivo.
    # Passa o valor 'utf-8' ao argumento 'encoding' para declarar a codificação.
```

```

# Guarda o resultado na variável 'text'.
text = file.read_text(encoding='utf-8')

# Aplica a expressão regular definida acima para remover a pontuação
# excessiva do texto. Guarda o resultado na variável 'mod_text'.
mod_text = punct.sub(' ', text)

# Define um novo nome de arquivo com o prefixo 'mod_' criando uma nova string.
# O objeto Path guarda o nome do arquivo como string no atributo 'name'.
# Combina as duas strings com o operador '+'.
new_filename = 'mod_' + file.name

# Define um novo objeto Path que aponta para o novo arquivo. O objeto Path
# junta automaticamente o diretório e o nome do arquivo para nós.
new_path = Path('data', new_filename)

# Imprime uma mensagem de status usando formatação de string. Ao adicionar
# o prefixo 'f' a uma string, podemos usar chaves {} para inserir uma
# variável dentro da string. Aqui inserimos o caminho atual do arquivo.
print(f'Writing modified text to {new_path}')

# Usa o método write_text() para escrever o texto modificado de 'mod_text'
# no arquivo, com codificação UTF-8.
new_path.write_text(mod_text, encoding='utf-8')

```

```

Writing modified text to data/mod_WP_1990-08-10-25A.txt
Writing modified text to data/mod_NYT_1991-01-16-A15.txt
Writing modified text to data/mod_WP_1991-01-17-A1B.txt

```

Como se vê, os objetos `Path` oferecem dois métodos convenientes para trabalhar com arquivos de texto: `read_text()` e `write_text()`. Eles permitem ler e gravar texto **sem** a instrução `with` — e, assim como o `with`, o arquivo apontado pelo `Path` é fechado automaticamente após a leitura.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Qual formato de texto é o mais interoperável para programação?

1. Texto formatado (*rich text*)
2. Texto estruturado (*structured text*)
3. Texto simples (*plain text*) ✓

2. Num texto, a sequência de dois caracteres “barra-n” representa:

1. Um espaço em branco
2. Uma quebra de linha ✓
3. Uma tabulação

3. O UTF-8 é retrocompatível com qual codificação?

1. ASCII ✓
2. Latin-1

3. UTF-16

4. Qual método de string divide o texto em uma lista?

1. `join()`
2. `split()` ✓
3. `replace()`

5. Numa expressão regular, o quantificador “dois ou mais” é escrito como:

1. `{2}`
2. `{2,}` ✓
3. `{,2}`

6. Qual método da classe `Path` coleta arquivos por um padrão com curinga (ex.: todos os `.txt`)?

1. `glob()` ✓
2. `read_text()`
3. `exists()`

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **distinguir** texto formatado, estruturado e simples, e a entender por que o texto simples é o formato preferido em programação;
2. **compreender** a codificação de caracteres (ASCII e Unicode/UTF-8) e por que ela causa problemas;
3. **carregar** arquivos com `open()` + `with` + `read()`, lidando com marcas como `e` e `\n`;
4. **manipular strings** com `replace()`, `split()`, `pop()` e `join()`, e montar um *pipeline* de substituições com listas, tuplas e laços `for`;
5. **definir padrões flexíveis** com expressões regulares (`re.compile()`, `sub()`, curingas, quantificadores e alternativas);
6. **processar vários arquivos** de uma vez com `pathlib.Path`, `glob()`, `read_text()` e `write_text()`.

Exercícios sugeridos

1. Carregue um dos arquivos do diretório `data` e conte quantas quebras de linha (`\n`) ele contém antes de qualquer limpeza.
2. Escreva um *pipeline* (lista de tuplas) que padronize **travessões** (`-`, `-`, `-`) em um único caractere.
3. Crie uma expressão regular que remova sequências de dois ou mais espaços em branco, substituindo-as por um único espaço.
4. Adapte o laço da Seção 5.3 para gravar os arquivos modificados em um novo diretório chamado `output` em vez de `data`.

Referências

- Rögnavaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Hill, M. J.; Hengchen, S. (2019). Quantifying the impact of dirty OCR on historical text analysis. *Digital Scholarship in the Humanities*.